Image Dithering: Notes

Xavier Doni

October 14, 2026

Contents

1 What is Dithering? **2**
1.1 Quantisation . . . 2

2 Historical Context **3**

3 Use Cases: Early Game Graphics **3**

4 The Primitive Approach: Nearest Colour **3**

5 Dithering Algorithms **4**
5.1 Error diffusion . . . 4
5.2 Floyd–Steinberg . . . 4
5.3 Atkinson . . . 4
5.4 Jarvis–Judice–Ninke . . . 4
5.5 Sierra . . . 4
5.6 Comparison . . . 5

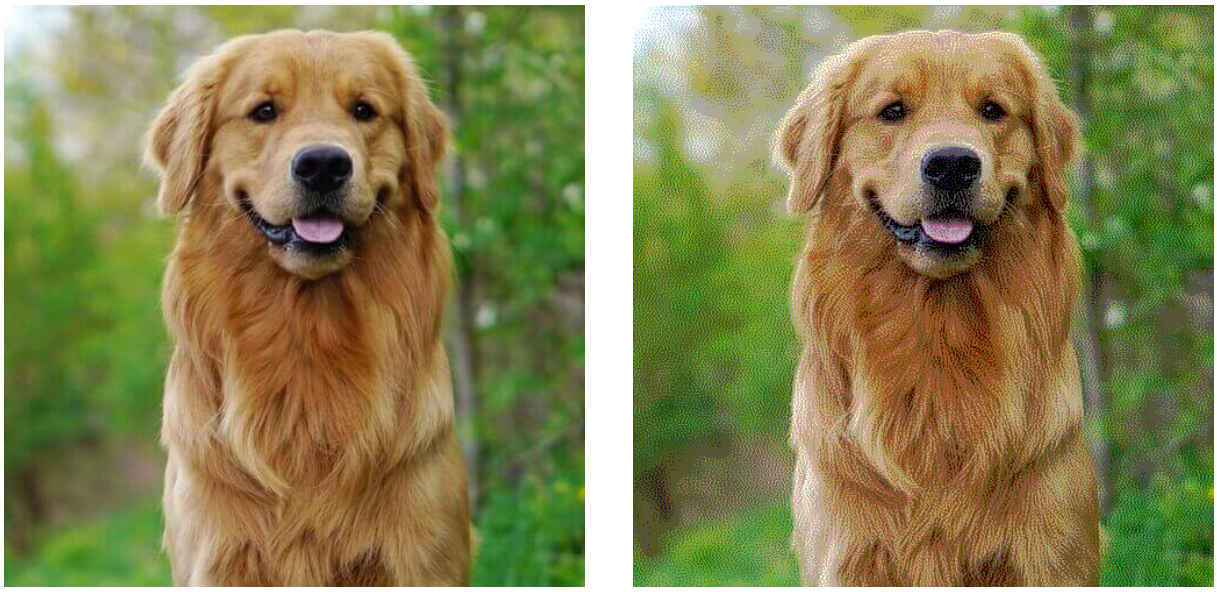
6 Conclusion **5**

A Dithered Image Gallery **5**

1 What is Dithering?

1.1 Quantisation

A digital image stores each pixel as a value from a limited set of intensities. In the real world, light is continuous, and the human eye can perceive millions of colours. Whereas the display (or file format) can only represent a *reduced* palette, every pixel must be mapped to one of the available values. This is known as *quantisation*.



(a) Original (b) Dithered

Figure 1: Original versus dithered

Dithering is the family of techniques that arranges the available values spatially so that, viewed from a distance, the eye averages neighbouring pixels and perceives intermediate tones that the palette cannot represent directly. Dither is routinely used in processing of both digital audio and visual data, and is a key technique in computer graphics.

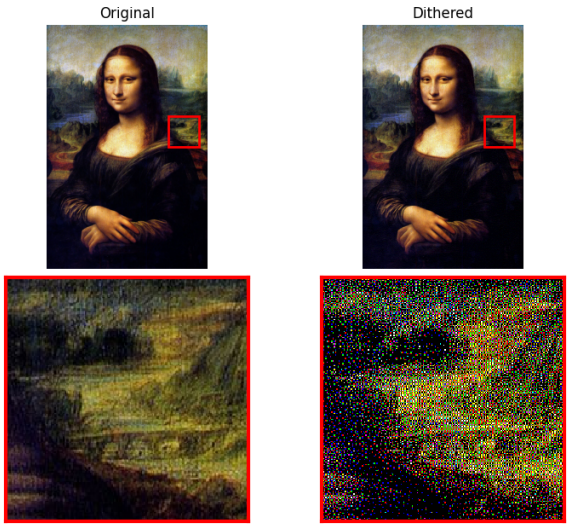


Figure 2: Dithering of the Mona Lisa. The original image (left) is quantised to 2 levels per channel and then dithered (right).

Let a single channel of an image be a function $I(x, y) \in [0, 255]$. A quantiser with L levels snaps each value to the nearest of L evenly spaced values:

$$\Delta = \frac{255}{L - 1}, \qquad q(v) = \Delta \cdot \operatorname{round}\!\left(\frac{v}{\Delta}\right). \tag{1}$$

The *quantisation error* of a pixel is $e = v - q(v)$, and always satisfies $|e| \le \Delta/2$. For a colour image the quantiser is applied to the red, green and blue channels independently, giving L^3 possible colours (Table 1).

Table 1: Palette size as a function of levels per channel L.

L	Grayscale shades	Colours (L^3)	Step Δ
2	2	8	255
3	3	27	127.5
4	4	64	85
6	6	216	51
256	256	16,777,216	1

2 Historical Context

Early display hardware could show only a handful of colours at once. The IBM Colour Graphics Adapter (CGA), introduced in 1981 with the IBM PC, is the classic example. Its 16-colour palette is built from four on/off signals: red, green, blue and an intensity bit (RGBI). Sixteen colours need only four bits per pixel, since $2^4 = 16$.

This should not be confused with "16-bit colour" (High Color), which uses 16 bits per pixel and yields $2^{16} = 65{,}536$ colours (Table 2).

Table 2: Bits per pixel versus number of distinct colours.

Name	Bits per pixel	Colours
CGA palette	4	16
"16-bit" High Color	16	65,536
24-bit True Color	24	16,777,216

In CGA's graphics modes, only four of the sixteen colours could be shown simultaneously, chosen from fixed palettes. Programmers therefore mixed pixel patterns to imitate the missing colours, which is a major reason dithering became widespread.

Add references for CGA specifications.

3 Use Cases: Early Game Graphics

Console and arcade artists faced the same constraint: a small palette and low resolution. The Sega Mega Drive (Genesis) offered a 9-bit palette of 512 colours, of which only a limited number could be shown at once. Artists in games such as *Sonic the Hedgehog* used checkerboard patterns of two colours; on a CRT television connected by composite video, the signal blurred the pattern into an apparent third colour or a semi-transparent effect (a commonly cited example is the waterfalls of Green Hill Zone).

Dithering also served monochrome displays: Atkinson's algorithm (Section 5.3) was written for the original Apple Macintosh, whose screen was strictly one bit per pixel.

Verify details and cite a source.

4 The Primitive Approach: Nearest Colour

The simplest strategy maps every pixel independently to the closest palette colour. With palette P and pixel $p = (r, g, b)$,

$$\operatorname{map}(p) = \arg\min_{c \in P} \sqrt{(r - c_r)^2 + (g - c_g)^2 + (b - c_b)^2}. \tag{2}$$

For two grayscale levels this reduces to thresholding at 128.

Remark (choice of distance). Euclidean distance is used rather than cosine similarity. Cosine similarity compares only the *direction* of two colour vectors and ignores their magnitude, so $(255, 0, 0)$ and $(128, 0, 0)$ would be considered identical even though one is much darker.

Nearest-colour mapping discards the error entirely. Smooth gradients collapse into flat regions with hard edges (*banding* or *contouring*), and large areas of similar tone become a single solid colour. The algorithms in Section 5 fix this by *remembering* the error.

Insert a threshold-versus-dithered comparison figure.

5 Dithering Algorithms

5.1 Error diffusion

All four algorithms below are *error-diffusion* methods. Pixels are processed in scan order. For the current pixel at (x, y):

$$\text{old} = I(x, y), \qquad \text{new} = q(\text{old}), \qquad e = \text{old} - \text{new}, \tag{3}$$

$$I(x + d_x,\, y + d_y) \mathrel{+}= \frac{w}{D}\, e \quad \text{for each } (d_x, d_y, w) \text{ in the kernel.} \tag{4}$$

The kernel lists which not-yet-processed neighbours receive a share of the error, with integer weights w and a divisor D. In the matrices below, $*$ marks the current pixel and $\cdot$ marks pixels that are never modified.

Our implementation uses *serpentine* scanning: the scan direction alternates every row (and the kernel is mirrored accordingly), which avoids the diagonal "worm" artefacts of always scanning left to right.

5.2 Floyd–Steinberg

Published by Floyd and Steinberg in 1976, this is the most widely used algorithm. It spreads the full error over four neighbours:

$$\frac{1}{16}\begin{pmatrix} \cdot & * & 7 \\ 3 & 5 & 1 \end{pmatrix} \tag{5}$$

5.3 Atkinson

Developed by Bill Atkinson at Apple for the original Macintosh, this kernel spreads the error over six neighbours but divides by 8, so only $6/8 = 75\%$ of the error is propagated:

$$\frac{1}{8}\begin{pmatrix} \cdot & * & 1 & 1 \\ 1 & 1 & 1 & \cdot \\ \cdot & 1 & \cdot & \cdot \end{pmatrix} \tag{6}$$

Discarding a quarter of the error gives higher contrast: highlights and shadows tend to saturate to pure white and black, while mid-tones keep crisp, less noisy detail.

5.4 Jarvis–Judice–Ninke

This kernel spreads the error over twelve neighbours across three rows:

$$\frac{1}{48}\begin{pmatrix} \cdot & \cdot & * & 7 & 5 \\ 3 & 5 & 7 & 5 & 3 \\ 1 & 3 & 5 & 3 & 1 \end{pmatrix} \tag{7}$$

The wider spread produces smoother results with finer texture, at the cost of more computation per pixel.

5.5 Sierra

The three-row Sierra kernel is a lighter relative of Jarvis–Judice–Ninke, using ten neighbours and a power-of-two divisor:

$$\frac{1}{32}\begin{pmatrix} \cdot & \cdot & * & 5 & 3 \\ 2 & 4 & 5 & 4 & 2 \\ \cdot & 2 & 3 & 2 & \cdot \end{pmatrix} \tag{8}$$

Table 3: Summary of the error-diffusion kernels used.

Algorithm	Neighbours	Divisor D	Error propagated
Floyd–Steinberg	4	16	100%
Atkinson	6	8	75%
Jarvis–Judice–Ninke	12	48	100%
Sierra	10	32	100%

5.6 Comparison

Verify publication years and add

6 Conclusion

Summarise your findings. Observations from the analysis tool that may be worth confirming on real images: (1) a dithered image's histogram collapses onto L values per channel, yet it still looks smooth because the tone is carried by the *density* of the dots; (2) error diffusion preserves the mean brightness almost exactly; (3) pixelising before dithering makes the pattern visible and keeps computation fast.

A Dithered Image Gallery

Table 4: Summary of all example images used.

Source	Size	Colours	Gray lvls	Mean	Std	Min	Max
Apple.png	2040x2245	538,569	249	156.37	85.12	7	255
Banana.jpg	3300x3300	58,356	255	246.24	19.66	1	255
Blackwhite-Wallpaper.jpg	1920x1080	256	256	113.16	111.32	0	255
Bruno-Mars.jpg	916x639	124,696	255	73.26	72.85	1	255
Canelo.jpg	451x680	36,513	244	73.80	55.71	0	246
Cat.jpg	554x554	90,238	246	118.83	57.10	0	251
Chun-Li.png	820x523	73,098	256	191.30	89.01	0	255
Eagle.jpg	1536x1155	138,380	256	136.80	60.89	0	255
Emoji.png	1024x1024	38,921	215	197.30	54.31	41	255
Flowers.jpg	1920x1200	207,874	243	109.21	61.14	0	243
GitHub.jpg	474x484	506	256	212.56	79.19	0	255
Golden-Retriever.jpeg	500x500	94,919	256	118.76	37.86	0	255
Greek-Statue.png	1024x1024	53,965	255	63.58	59.70	0	254
Iron-Man.jpg	840x930	46,620	256	220.40	63.64	0	255
KFC-Logo.png	1219x914	14,477	236	152.23	92.63	20	255
Lanterns.jpg	2160x1080	76,346	256	101.17	57.59	0	255
Mario.jpg	1350x1080	206,464	256	195.13	79.48	0	255
Mona-Lisa.jpg	1057x1600	272,610	256	60.15	64.78	0	255
Parade.jpg	1500x1000	527,062	256	139.82	63.42	0	255
Pen.jpg	1000x1000	30,819	256	240.88	46.05	0	255
Pokémon-Legends-ZA.jpg	1424x800	158,072	256	75.48	45.46	0	255
Rock.png	1280x960	166,146	231	181.19	63.13	25	255
Samus.png	1529x1034	159,799	256	214.27	74.93	0	255
Sonic.png	1209x1677	16	15	183.07	92.06	0	255
Sphere.jpg	630x630	23,319	210	143.77	70.64	44	255
Starry-Night.jpg	565x353	133,414	256	110.87	66.03	0	255
Tennis-Balls.jpg	1600x900	71,200	235	182.29	36.45	1	236
WiFi.jpg	474x484	10,549	213	194.27	80.25	42	255
Windows-XP-wallpaper-4K.jpg	2560x1707	125,061	219	128.74	42.59	21	239
Yahoo.png	920x920	26,412	213	107.70	79.71	43	255

Run `generate_ga` to create `appendix_ga`